

Name - Saikat Sheet

Batch - 25SUB5936_Linux System Programming

ID - 68500

Segmentation & Paging

Advanced Principles of Memory Management and Address Translation in Modern
Operating Systems

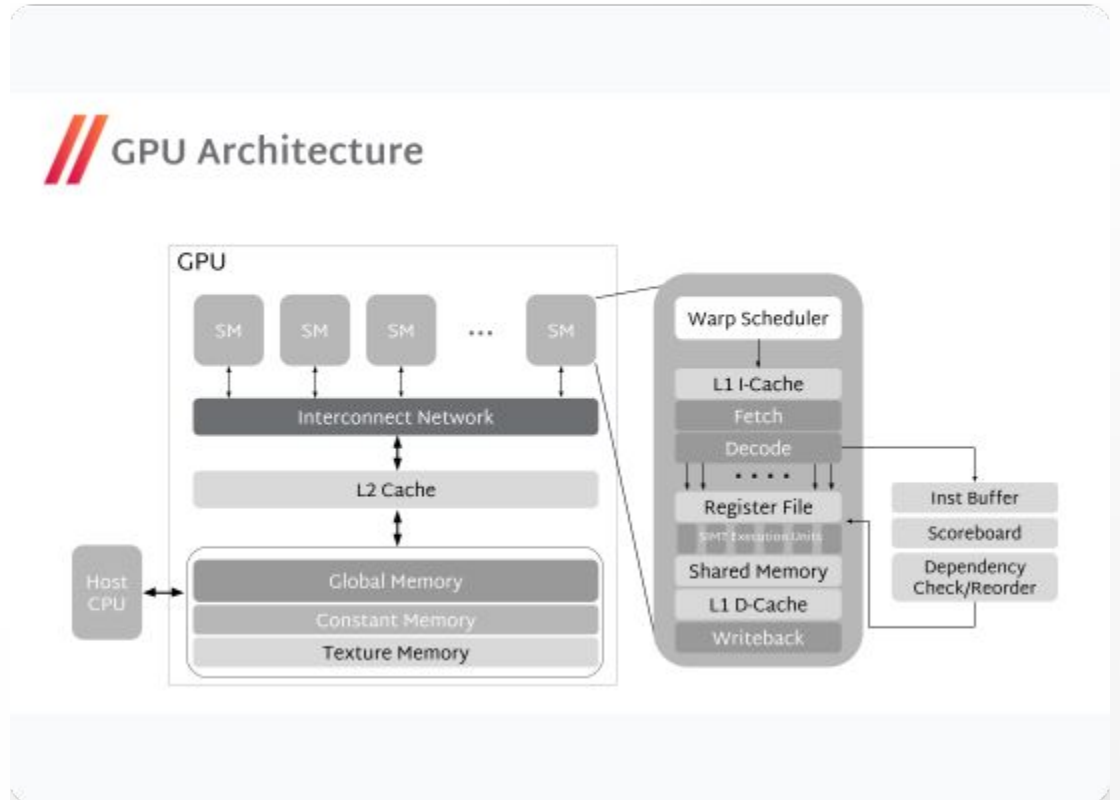
| Logical vs. Physical Space

The Mapping Layer

Modern CPUs generate **Logical Addresses** (Virtual Space) which provide a clean, contiguous view for programmers.

The **MMU (Memory Management Unit)** translates these into **Physical Addresses** corresponding to actual RAM locations.

This abstraction enables multitasking, protection, and efficient hardware utilization.



| Segmentation: Logical Partitioning



Code

Read-only segment containing executable instructions. Optimized for instruction cache efficiency.



Data

Variable-sized blocks storing global variables and static arrays. Supports logical isolation.

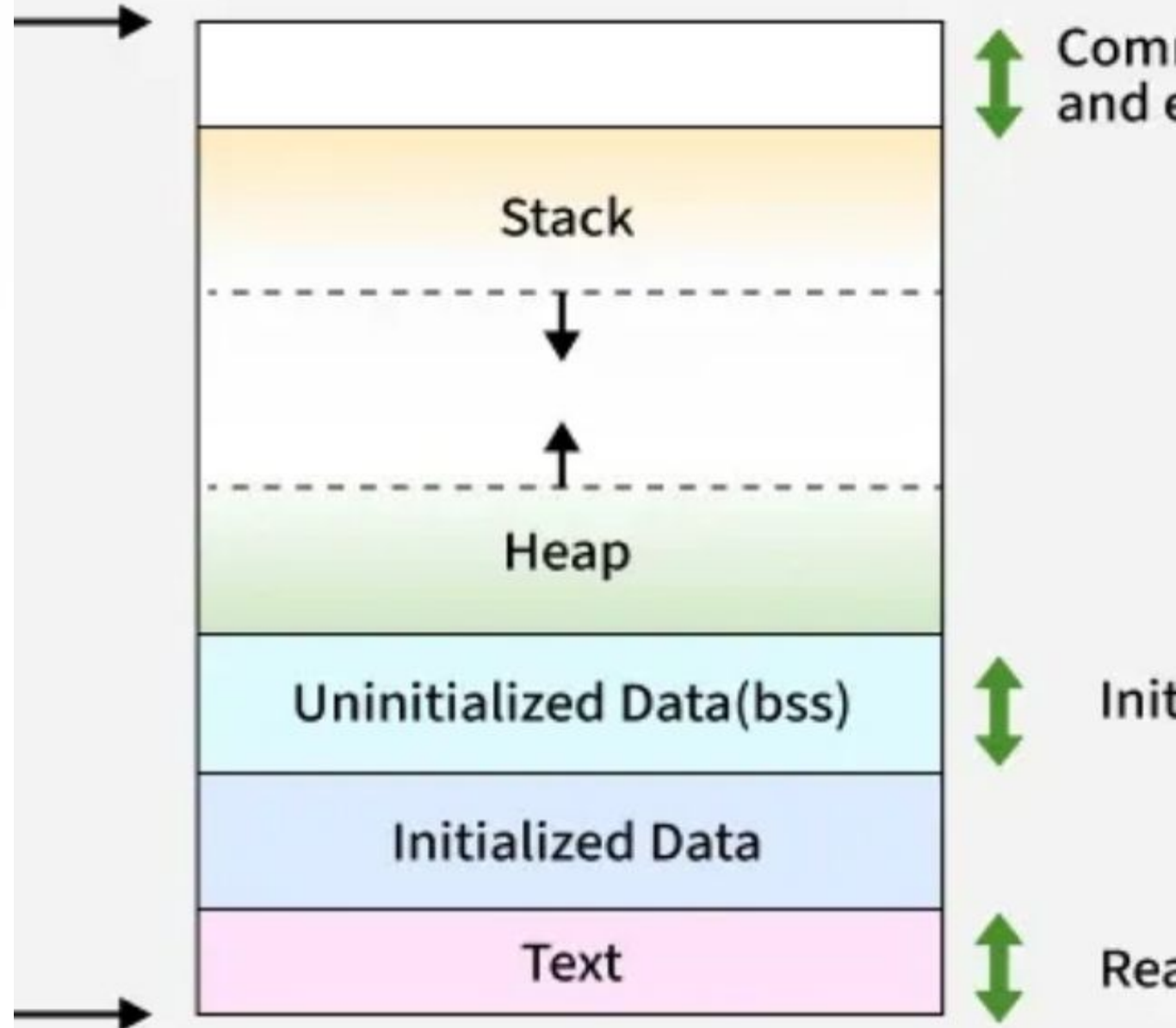


Stack/Heap

Dynamically growing segments for local variables and dynamic memory allocation.

Visualizing Segmentation

Segmentation reflects the **Programmer's View** of memory, where a program is a collection of logical units. Each segment has a different size, leading to the challenge of **External Fragmentation**: gaps in physical memory that are too small for new segments. Segments are non-contiguous in RAM but logically coherent.



| Segment Address Translation

Base + Offset

Physical Address Formula

The Segment Table

The logical address consists of .

$$A_{\text{phys}} = \text{Base}_{\text{seg}} + \text{Offset}$$

The system verifies that $\text{Offset} < \text{Limit}$ to prevent segmentation faults (Illegal memory access).

| Paging: Fixed-Block Allocation

Uniform Division

Physical memory is broken into fixed-size blocks called **Frames**. Logical memory is broken into blocks of the same size called **Pages**.

Efficiency Advantages

Eliminates external fragmentation entirely.

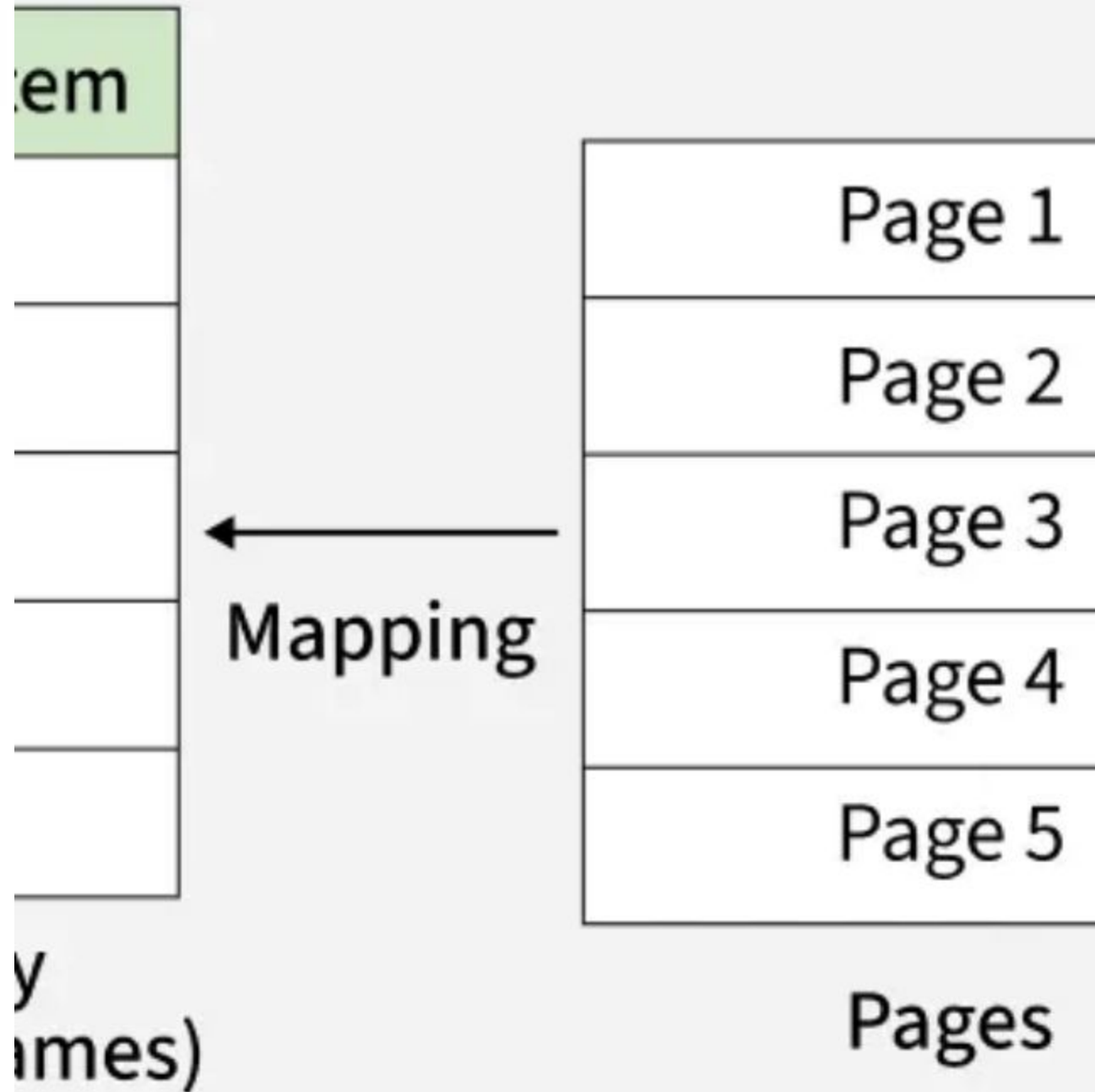
Memory allocation is simple: any free frame can store any page. Managed entirely by the OS.

Visualizing Paged Memory

Paging decouples logical contiguous space from physical scattered space. A **Page Table** tracks the mapping.

While external fragmentation is removed, **Internal Fragmentation** occurs if the process does not fill the last page entirely.

Fixed sizes simplify hardware management and swapping to disk.



Paging Address Translation

Frame # & Offset

Physical Mapping

Page Table Lookup

Logical address consists of .

$$A_{\text{phys}} = (f \times S) + d$$

Where f is the Frame #, S is Page Size, and d is the Offset.

Hardware often uses bit-shifting for zero overhead.

Comparison: Paging vs. Segmentation

Feature	Paging	Segmentation
Unit Size	Fixed-size blocks (Hardware defined)	Variable-sized segments (User/Logic defined)
Fragmentation	Internal fragmentation (last page)	External fragmentation (unused gaps)
Visibility	Transparent to the programmer	Visible to the programmer/compiler
Sharing/Protection	Difficult (requires page alignment)	Easy (maps to logical units)

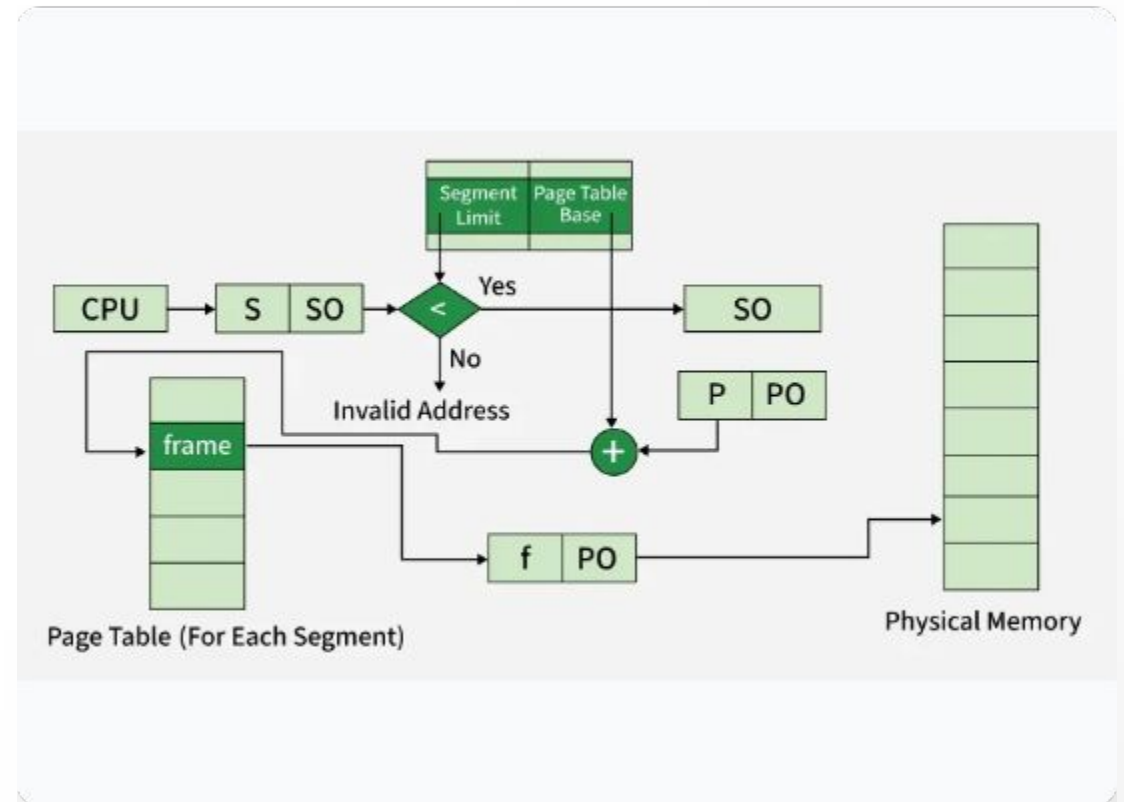
Hybrid Approach: Segmented Paging

The Best of Both Worlds

Modern x86 architectures divide memory into logical **Segments**, and then divide those segments into fixed **Pages**.

Benefits: Logical protection of segmentation combined with the physical flexibility of paging. No external fragmentation.

Logical Address: .



*Thank
you!*